

Data science Technical Report

Abstract

This project presents an **AI-powered FIFA Career Mode decision-support system** designed to help users make smarter decisions while managing a virtual football club. FIFA Career Mode involves several connected decisions, including simulating matches, choosing player positions, evaluating player quality, predicting future performance, assessing injury risk, and deciding whether a transfer is financially and tactically worthwhile. The problem addressed by this project is that these decisions are often made based on visible ratings, market value, or personal judgment, without deeper data-driven analysis.

The system combines multiple machine learning modules into one integrated architecture. The Match Simulation Engine predicts match-related outcomes such as scorelines, winners, and scoring events. The Player Development and Squad Planning Engine predicts a player's best position, estimates his overall rating from his attributes, and recommends statistically similar players who can act as replacements or alternative transfer targets. The Transfer Decision Engine predicts injury risk, future goals and assists, and transfer market value to help classify signings as good, risky, overpriced, or worth further review.

The results showed strong performance across several modules. Logistic Regression achieved **82.82% accuracy** for position classification, while Polynomial Regression predicted overall rating with $R^2 = 0.9801$ and $MSE = 0.962$. The injury classifier achieved **95.0% accuracy**, and the scouting report generator successfully converted predictions into a final transfer recommendation. Overall, the project demonstrates how machine learning can improve Career Mode decisions by combining prediction, evaluation, and recommendation into a single intelligent system.

1. Introduction

1.1 Background and Context

Football management games such as **FIFA Career Mode** require users to make many decisions that resemble real club management. A career-mode manager must choose lineups, assign player positions, simulate matches, buy and sell players, monitor player development, manage injury risk, and decide whether a transfer is worth the money. These decisions are connected: a player may have high potential but poor injury reliability, strong market value but weak future performance, or good attributes but an unsuitable assigned position.

In traditional Career Mode gameplay, many decisions depend mainly on visible information such as overall rating, age, market value, position, and recent performance. However, these values do not always provide enough insight. For example, a player with a high listed market value may not be worth signing if his predicted future output is low or his injury risk is high. Similarly, a player may perform better in a different position than the one originally assigned to him. This creates a need for a more analytical decision-support system that evaluates players and matches using multiple data-driven perspectives.

This project addresses that need by building an **AI-powered FIFA Career Mode assistant**. The system combines several machine learning models that support three major areas of career-mode management:

- **Match Simulation Engine:** predicts match outcomes, scorelines, and scoring-related events.
- **Player Development and Position Engine:** predicts a player's best position, estimates his overall rating from his attributes, and recommends similar players who can be used as replacements or alternative transfer targets.
- **Transfer Decision Engine:** predicts injury risk, future performance, and market value to support transfer decisions.

Together, these components help the user move from simple intuition-based decisions toward more objective, data-supported career-mode management.

1.2 Literature Review

Machine learning has become widely used in football analytics because football produces large amounts of structured data, including player attributes, match events, performance statistics, injury records, and market information. In football analysis, supervised learning models are commonly used for classification and regression problems, such as predicting match outcomes, estimating player performance, classifying player roles, or forecasting transfer value.

For player evaluation, classification algorithms can be used to predict a player's best position based on technical and physical attributes. In this project, Logistic Regression, K-Nearest Neighbors, and Gaussian Naive Bayes were compared for position classification. Logistic Regression produced the strongest result, showing that weighted combinations of player attributes can effectively separate football positions. Regression models are also useful for estimating player quality. Linear Regression and Ridge Regression provided strong baselines, while Polynomial Regression achieved the best performance by capturing non-linear relationships between attributes and overall rating.

In football scouting and transfer analysis, predictive models are often used to reduce uncertainty. A player's transfer value is affected by many factors, including performance, age, playing time, club context, injury history, and previous market value. Random Forest models are useful in this context because they can capture non-linear relationships between variables. Similarly, injury prediction is a classification problem where the goal is to identify risky players before they become unavailable. Since injury prediction may involve imbalanced classes, metrics such as precision, recall, and F1-score are important alongside accuracy.

Match analysis also plays an important role in football decision-making. Event data, such as shot location, angle, body part, and defensive pressure, can be used to estimate the probability of a shot becoming a goal. Match-level features can also be used to predict scorelines and winners. This type of modeling supports the match simulation part of the system, making it relevant to FIFA Career Mode, where users often simulate matches instead of manually playing every fixture.

Previous work in sports analytics shows that combining prediction, evaluation, and visualization can support better decision-making. This project applies that idea to FIFA Career Mode by integrating multiple models into one system that helps the user evaluate players, simulate matches, and make transfer decisions more intelligently.

1.3 Project Objectives

The main objective of this project is to build an **AI-powered FIFA Career Mode decision-support system** that helps users manage a virtual football club using machine learning predictions and analytical insights.

The specific objectives are:

1. **Develop a Match Simulation Engine**
To predict match-related outcomes such as expected scoreline, winner, shot outcomes, and scoring events, supporting the simulation of Career Mode fixtures.
2. **Develop a Player Development and Position Engine**
To predict a player's most suitable position and estimate his overall rating based on physical and technical attributes.
3. **Develop a Player Recommendation Module**
To identify statistically similar players based on their attribute profiles, helping the user find replacements, compare alternatives, and support squad planning when buying or selling players in Career Mode.
4. **Develop a Transfer Decision Engine**
To evaluate potential transfers using injury risk prediction, future goals and assists prediction, and transfer market value estimation.

5. **Support smarter transfer decisions**

To help classify a player as a good signing, risky signing, overpriced option, undervalued opportunity, or player requiring further investigation.

6. **Provide interpretable insights**

To use visualizations such as heatmaps, confusion matrices, radar charts, dashboards, and feature-importance plots to explain model behavior and support analysis.

7. **Generate decision-ready outputs**

To produce a scouting-style player report that summarizes predictions, scores, risk levels, market value, and final recommendations in a form that can be easily understood by the user.

8. **Combine separate models into one coherent system**

To avoid treating the project as isolated machine learning tasks and instead present it as a unified Career Mode assistant that supports squad management, player development, match simulation, and transfer strategy.

2. **Materials and Methods:**

1. **Data Description**

This project used multiple football-related datasets to support different components of the proposed **AI-powered FIFA Career Mode decision-support system**. The datasets cover match events, player attributes, injury risk, future performance, and transfer market value. Together, they allow the system to simulate career-mode matches, evaluate player positions and overall ratings, estimate injury and performance risk, suggest alternatives players, and support transfer decisions by judging whether a player is a good or risky signing.

1.1 **StatsBomb Open Data**

The first data source was the **StatsBomb open-data repository**, accessed using the statsbombpy Python library. This dataset contains real football match-event data from multiple competitions and seasons, including domestic leagues and international tournaments. Each match is represented as a sequence of timestamped events such as passes, shots, dribbles, and other in-game actions. The pitch coordinates follow the StatsBomb coordinate system, where the pitch is represented on a **120 × 80 grid**.

This dataset was used to build the **Match Simulation Engine** of the FIFA Career Mode system. This module supports match simulation by analyzing real match events, predicting shot outcomes, estimating scorelines, identifying the likely winner, and supporting match-related predictions such as likely goal scorers.

Key Variables

Variable	Type	Description
x, y	Numerical	Shot location on the StatsBomb pitch grid
minute	Numerical	Minute of the match when the shot occurred
shot outcome	Categorical	Result of the shot, such as goal or no goal
body_part	Categorical/Binary	Whether the shot was taken by head, foot, or other body part
under pressure	Binary	Whether the player was under defensive pressure
xG	Numerical	StatsBomb expected-goals value
home_score, away_score	Numerical	Final goals scored by home and away teams
home passes, away passes	Numerical	Total passes per team
home shots, away shots	Numerical	Total shots per team

Two cleaned CSV files were produced from this data:

Output File	Purpose
shots_clean.csv	Used by the shot goal/no-goal classification model
matches_clean.csv	Used by the match score and winner prediction model

The project pipeline was designed so that **data_cleaning.py** runs first, producing the cleaned datasets required by the modeling scripts.

1.2 EA Sports FC 24 Player Dataset

The second major dataset was an **EA Sports FC 24 Kaggle player dataset**. This dataset was used for player-level modeling, including position prediction, overall rating prediction, player recommendation, and explainability analysis.

The team initially worked with a smaller dataset containing 11,484 rows, but after investigation, it was discovered that 11,422 rows were duplicated, leaving only 62 unique players. The inflated dataset caused unrealistic model results, including a **KNN accuracy of 100%**, which was identified as a **data leakage** problem. The issue was caused partly by an **Unnamed: 0 column**

that acted as a unique index and prevented normal duplicate detection. After this discovery, the team pivoted to a larger full-scale EA Sports FC 24 dataset containing 18,773 players, which was reduced to 18,740 unique players after removing 33 duplicate records.

This dataset supported the **Player Development and Position Engine** of the FIFA Career Mode system.

Key Variables

Variable Group	Type	Description
Physical attributes	Numerical	Pace, acceleration, sprint speed, jumping, stamina, strength
Technical attributes	Numerical	Finishing, passing, dribbling, tackling, heading accuracy
Goalkeeper attributes	Numerical	GK diving, handling, kicking, reflexes, positioning
preferred_foot	Categorical/Binary	Encoded as left/right foot
best_position	Categorical	Target variable for position classification
overall_rating	Numerical	Target variable for player quality regression

The final selected feature set contained approximately 41 core physical and technical attributes, after removing non-predictive metadata such as URLs, images, descriptions, and club identifiers.

1.3 Injury Prediction Dataset

The injury module used the **University Football Injury Prediction Dataset from Kaggle**. This dataset was used to predict whether a player is likely to sustain an injury in the next season. The target variable was binary:

- 1 = Injury expected next season;
- 0 = No injury expected next season.

The injury dataset was used by the **Transfer Decision Engine** of the FIFA Career Mode system. Its purpose was to estimate a player's injury risk before completing a transfer, keeping him in the squad, or depending on him as a key player during the season.

Key Variables

Variable Group	Type	Description
----------------	------	-------------

Physical indicators	Numerical	Strength, fitness, and physical condition variables
Training-related variables	Numerical	Training intensity and workload-related features
Lifestyle/health indicators	Numerical	Sleep, nutrition, previous injuries, and related risk indicators
Injury_Next_Season	Binary	Injury-risk target variable

1.4 Player Performance Dataset

The player performance module used the **Top 5 League Player Stats 2017–2025 Kaggle** dataset. This dataset contains historical player statistics across the top five European leagues over multiple seasons. It was used to predict future attacking performance, specifically:

- goals in the next season;
- assists in the next season.

The data was sorted by player and season to enable time-aware feature engineering. The team created shifted target variables such as **goals_next** and **assists_next**, representing the following season’s output.

Key Variables

Variable	Type	Description
player	Categorical	Player name or identifier
season	Temporal/Categorical	Football season
league	Categorical	League name
team	Categorical	Player’s club
pos_	Categorical	Player position
Goals/assists stats	Numerical	Historical attacking output
goals_next	Numerical	Target variable for next-season goals
assists_next	Numerical	Target variable for next-season assists
goals_trend	Numerical	Difference between current and previous season goals

This dataset supported the **Transfer Decision Engine** of the FIFA Career Mode system. It was used to predict a player’s future goals and assists, helping evaluate whether a potential signing is likely to deliver strong performance in upcoming seasons.

1.5 Transfer Value Dataset

The transfer value module used the **Football Players Transfer Fee Prediction** Dataset from Kaggle. This dataset was used to estimate a player’s current market value using objective player statistics rather than media hype or subjective reputation. The target variable was **current_value**.

This dataset supported the **Transfer Decision Engine** of the FIFA Career Mode system. It helped evaluate whether a potential transfer is financially reasonable by comparing the predicted value with the player’s listed or expected market value, allowing the system to classify a signing as valuable, overpriced, risky, or worth further investigation.

Key Variables

Variable Group	Type	Description
Performance statistics	Numerical	Goals, assists, cards, clean sheets, and related match statistics
Physical/player data	Numerical	Age, appearances, minutes played, and other attributes
Club information	Categorical	Team or club affiliation
current_value	Numerical	Target variable for transfer market value prediction

2. Data Cleaning and Preprocessing

Because the system combines several independent datasets, preprocessing was essential to ensure that each model received clean, consistent, and meaningful input features.

2.1 StatsBomb Data Cleaning

The StatsBomb pipeline was handled by **data_cleaning.py**. This script performed three main tasks:

1. fetching raw match and shot data;
2. cleaning and engineering shot-level features;
3. cleaning and engineering match-level features.

For shot data, rows with missing coordinates were removed because distance and angle cannot be calculated without valid x and y values. Coordinates were clipped to remain within the StatsBomb pitch bounds: $x \in [0,120]$ and $y \in [0,80]$.

The shot cleaning pipeline engineered the following features:

Feature	Description
distance	Euclidean distance from shot location to the goal center
angle	Angle between the two goalposts from the shooter's position
body_part	Encoded as 1 for header and 0 for foot/other
under_pressure	Encoded as 1 if under pressure, otherwise 0
goal	Encoded as 1 if the shot resulted in a goal, otherwise 0
xG	Expected-goals value, filled using a fallback logistic formula when missing

The distance from goal was calculated as:

$$distance = \sqrt{(120 - x)^2 + (40 - y)^2}$$

The shot angle was calculated using the dot-product formula between vectors from the shooter to the two goalposts. Missing xG values were filled using a fallback logistic formula based on distance and angle. The resulting xG values were clipped between 0.01 and 0.98 to avoid unrealistic probability extremes.

For match data, rows with missing scores were removed, scores were converted to integers, and corrupt score outliers were clipped to the range [0,15]. Additional features were engineered, including:

- total goals;
- match outcome code;
- possession percentage;
- home and away team ratings;
- expected home goals;
- expected away goals.

Team ratings were joined using a hardcoded lookup table containing attack, defense, midfield, form, average goals, and average conceded values for more than 30 teams. Unknown teams were assigned default values.

2.2 EA Sports FC 24 Player Data Cleaning

The player dataset required special attention because the first version contained severe duplication. The team discovered that a dataset with 11,484 rows actually contained **only 62 unique players**, because 11,422 rows were repeated copies. This caused unrealistic machine learning results, especially for KNN, where duplicated players appeared in both training and testing sets.

The hidden issue was an **Unnamed: 0** column, which acted as a unique row identifier. Since every duplicated row had a different index value, standard duplicate removal failed. The team solved this by dropping duplicates based only on actual player attributes rather than artificial row identifiers.

After identifying this data integrity problem, the project moved to a larger and more reliable EA Sports FC 24 Kaggle dataset. The final preprocessing steps included:

- removing 33 duplicate records;
- dropping non-predictive metadata such as URLs, images, descriptions, and club IDs;
- selecting 41 core numerical player attributes;
- verifying that the selected numerical features had no missing values;
- binary encoding preferred_foot, where Left = 0 and Right = 1;
- scaling features where required for distance-based algorithms.

This preprocessing allowed the player models to be trained on **18,740 unique players**, producing more realistic and generalizable results.

2.3 Injury Dataset Preprocessing

The injury dataset was cleaned and prepared using the following steps:

- duplicate records were removed;
- the categorical Position column was dropped after analysis showed limited predictive value;
- missing numerical values were imputed using column means;
- numerical features were normalized using StandardScaler;
- A correlation heatmap was generated to identify relationships between input features and the Injury_Next_Season target.

The scaling step was included inside a scikit-learn pipeline to ensure preprocessing was applied correctly during both cross-validation and final model training.

2.4 Performance Dataset Preprocessing

The performance dataset required time-aware preprocessing because the goal was to predict **future performance from previous seasons**. The dataset was loaded using a semicolon delimiter and comma decimal format. It was then sorted by player and season to create future target variables.

The main preprocessing steps were:

- creating goals_next as the player's goals in the following season;
- creating assists_next as the player's assists in the following season;
- creating goals_trend as the difference between current and previous season goals;
- encoding positions using an ordinal map: GK = 1, DF = 2, MF = 3, FW = 4;
- dropping non-predictive identifiers such as player, pos_, league, team, and nation_;
- selecting the top 25 most correlated features for each target variable;
- using a temporal train/test split, with seasons before 2023/24 used for training and the 2023/24 season used for testing.

The temporal split was important because it prevented future data from leaking into the training process.

2.5 Transfer Value Dataset Preprocessing

The transfer value dataset was cleaned and transformed to allow fair comparison between players with different playing time. The key preprocessing steps were:

- deduplicating records based on player identifier;
- checking and reporting missing values;
- confirming that the selected dataset was clean;
- converting raw counting statistics to per-90-minute rates;
- applying target encoding to the team variable;
- dropping non-predictive raw string columns such as name, player, team, and position;
- standardizing features for correlation analysis;
- removing low-signal features with absolute correlation below 0.05.

Per-90 normalization was applied using the formula:

$$\text{per_90_stat} = (\text{stat} \times 90) / \text{minutes_played}$$

This transformation made comparisons fairer because players with fewer minutes were not automatically penalized against players who had played more. Target encoding was used for the team variable by replacing each team with the mean transfer value of its players, giving the model a compact club-level signal without creating too many one-hot encoded columns.

3. Methodology

The methodology was designed around a **multi-model FIFA Career Mode architecture**. Each model represents a specialized component of the career-mode assistant, supporting a different management decision such as match simulation, player position evaluation, overall rating prediction, injury-risk estimation, future performance forecasting, and transfer value assessment. Together, these models help the user make more informed decisions when managing a squad, simulating matches, developing players, and evaluating potential transfers.

3.1 Shot Outcome Classification

The shot outcome model predicts whether a shot will result in a goal. This is a binary classification task:

- 1 = Goal;
- 0 = No Goal.

Model Used

A **Decision Tree Classifier** was used for this task.

Justification

A Decision Tree was suitable because shot success is not always linearly related to the input variables. For example, distance, angle, body part, and pressure interact in non-linear ways. A close-range shot from a central angle has a much higher chance of becoming a goal than a long-range shot from a narrow angle, and a tree model can naturally capture these threshold-based decision rules. The model is also interpretable because its decision paths and feature importances can be inspected.

Input Features

Feature	Type	Description
distance	Numerical	Distance from the shot location to the goal center
angle	Numerical	Shooting angle to goal
body_part	Binary	Header or non-header
under_pressure	Binary	Whether the shooter was under pressure

Target

Target	Type	Description
goal	Binary	Whether the shot resulted in a goal

Training Procedure

The model loaded data from **shots_clean.csv**. Rows with missing values in selected features or target were removed. If fewer than 100 valid rows were available, the script generated a fallback synthetic dataset of 5,000 shots using the same logistic xG logic used in the cleaning pipeline. The data was split using an 80/20 stratified train-test split, preserving the class distribution in both sets. The Decision Tree used **max_depth = 7**, **min_samples_split = 20**, and **random_state = 42**.

Evaluation Metrics

The model was evaluated using:

- accuracy;
- precision;
- recall;
- F1-score;
- confusion matrix;
- feature importance.

3.2 Match Score and Winner Prediction

The match prediction model estimates home goals, away goals, and the final winner.

Model Used

Two separate Linear Regression models were trained:

- one model for **home_score**;
- one model for **away_score**.

Justification

Separate models were used because home and away goals are two different scoring processes. The number of goals scored by the home team may be affected differently by the same features compared with the away team. Training two models prevents the system from incorrectly treating both outputs as one combined target.

Input Features

Feature	Type	Description
exp_home_goals	Numerical	Expected scoring rate for the home team
exp_away_goals	Numerical	Expected scoring rate for the away team

Targets

Target	Type	Description
home_score	Numerical	Actual home goals
away_score	Numerical	Actual away goals

Winner Derivation

After predicting home and away goals, the model calculated:

$$\text{goal_difference} = \text{predicted_home_goals} - \text{predicted_away_goals}$$

The predicted winner was derived using the following rule:

Condition	Predicted Outcome
$\text{goal_difference} > 0.15$	Home Win
$-0.15 \leq \text{goal_difference} \leq 0.15$	Draw
$\text{goal_difference} < -0.15$	Away Win

The ± 0.15 draw band was used to avoid forcing very close predicted scorelines into home or away wins.

Evaluation Metrics

The model was evaluated using:

- R^2 for home goals;
- RMSE for home goals;
- R^2 for away goals;
- RMSE for away goals;

- winner prediction accuracy.

3.3 Player Position Classification

The player position model predicts a player's best position based on physical and technical attributes.

Models Compared

Three classification algorithms were compared:

Model	Purpose
Logistic Regression	Linear probabilistic classifier for multi-class position prediction
K-Nearest Neighbors	Distance-based classifier using similarity between player profiles
Gaussian Naive Bayes	Probabilistic classifier based on feature independence assumption

Final Selected Model

Logistic Regression was selected as the primary position classification model because it achieved the highest reported accuracy, **82.82%**, on the holdout test set.

Justification

Logistic Regression performed well because football positions can often be separated through weighted combinations of technical and physical attributes. For example, attacking roles are strongly related to finishing, dribbling, acceleration, and attacking positioning, while defensive roles are related to tackling, marking, strength, and interceptions.

KNN also performed reasonably, reaching **70.25%** accuracy, but its performance was weaker in high-dimensional space. Gaussian Naive Bayes achieved **53.71%** accuracy, mainly because its **independence assumption is unrealistic for player attributes**. Many football attributes are naturally correlated, such as sprint speed and acceleration or jumping and heading accuracy

Evaluation Metrics

The model was evaluated using:

- accuracy;
- confusion matrix.

3.4 Overall Rating Prediction

The overall rating model predicts a player's overall quality score based on his technical and physical attributes.

Models Compared

Model	Purpose
Linear Regression	Baseline linear relationship between attributes and rating
Ridge Regression	Linear regression with L2 regularization
Polynomial Regression, Degree 2	Captures non-linear and interaction effects between attributes

Final Selected Model

Polynomial Regression with degree 2 was selected as the optimal model. It achieved:

- $R^2 = 0.9801$
- $MSE = 0.9623$

on the unseen test set.

Justification

Linear Regression and Ridge Regression both achieved strong results, with R^2 around **0.8941** and MSE around **5.127–5.128**. However, Polynomial Regression performed significantly better because overall player ratings may depend on interactions between attributes rather than simple independent linear effects. For example, the combination of sprint speed and acceleration, or ball control and dribbling, can be more meaningful than each attribute alone.

Evaluation Metrics

The rating prediction models were evaluated using:

- R^2 score;
- Mean Squared Error;
- cross-validation stability for Ridge Regression.

3.5 Player Recommendation System

The player recommender system was designed to identify statistically similar players. This is useful when a club wants to replace a player being sold, compare scouting options, or find a cheaper alternative with similar attributes.

Model Used

The system used **NearestNeighbors** from scikit-learn.

Justification

Nearest Neighbors is suitable for similarity-based recommendation because it compares players directly in a multidimensional attribute space. Each player is represented as a vector of technical and physical attributes. The model then finds players with the smallest Euclidean distance from the target player.

Preprocessing

All core attributes were normalized using StandardScaler. This step was necessary because features with larger numerical ranges could otherwise dominate the Euclidean distance calculation. After scaling, every attribute contributed more fairly to the similarity search.

Output

The recommender returns the top five most statistically similar players to a selected target player.

3.6 Explainable AI for Player Rating

Although Polynomial Regression produced the best rating prediction performance, its equation is difficult to interpret. To explain which player attributes contribute most strongly to high ratings, the team trained a secondary explainability model.

Model Used

A **Random Forest Regressor** was trained to predict **overall_rating**.

Justification

Random Forest models are useful for explainability because they provide feature importance values. These values show which variables most frequently and effectively reduce prediction error across the trees.

Output

The explainability model identified **Reactions** as the most important feature for predicting overall rating, accounting for approximately 75.9% of the model's decision weight. Ball control was also identified as an important factor. A horizontal bar chart was generated to visualize the top 15 most influential features.

3.7 Injury Risk Classification

The injury risk model predicts whether a player is likely to suffer an injury in the next season.

Model Used

A **Logistic Regression** classifier was used inside a scikit-learn pipeline:

StandardScaler → LogisticRegression

Justification

Logistic Regression was appropriate because the task is binary classification and the output can be interpreted as an injury-risk probability. The model is also relatively interpretable compared with more complex classifiers. Because injury data can be imbalanced, the model was tuned using F1-score rather than accuracy.

Hyperparameter Tuning

The model was tuned using **GridSearchCV** with 5-fold cross-validation.

Parameter	Values Tested
C	0.01, 0.1, 1, 10
penalty	L2
class_weight	None, balanced
max_iter	10,000

Evaluation Metrics

The injury classifier was evaluated using:

- train accuracy;
- test accuracy;
- precision;
- recall;
- F1-score;
- confusion matrix.

3.8 Future Performance Prediction

The performance prediction module estimates a player's next-season goals and assists.

Models Used

Two independent **Random Forest Regressor** models were trained:

- one model for goals_next;
- one model for assists_next.

Justification

Random Forest Regression was selected because football performance is affected by non-linear relationships between many variables, such as position, previous performance, playing time, and trends across seasons. Random Forests can capture these non-linear interactions without requiring strict linear assumptions.

Hyperparameter Tuning

The models were tuned using **GridSearchCV** with 5-fold cross-validation.

Parameter	Values Tested
n_estimators	150, 200, 300
max_depth	8, 12, None
min_samples_split	2, 5

Train/Test Strategy

A temporal split was used:

- training data: seasons before 2023/24;
- test data: 2023/24 season.

This was selected instead of a random split to prevent future-season information from leaking into training.

Evaluation Metrics

The models were evaluated using:

- train R^2 ;
- test R^2 ;
- feature correlation heatmaps for each target.

3.9 Transfer Market Value Prediction

The transfer value model predicts a player's current market value from statistical and physical features.

Model Used

A **Random Forest Regressor** was used.

Justification

Transfer value is influenced by many non-linear factors, including performance, age, club strength, playing time, and injury history. Random Forest Regression was suitable because it can model complex relationships and interactions without requiring a simple linear formula.

Hyperparameter Tuning

The model was tuned using **GridSearchCV** with 5-fold cross-validation.

Parameter	Values Tested
n_estimators	150, 200
max_depth	6, 8

Train/Test Strategy

An 80/20 random train-test split was used with random_state = 42.

Evaluation Metrics

The model was evaluated using:

- R^2 score on the test set;
- correlation heatmap against current_value.

3.10 Automated Player Scouting Report Generator

The project also included an automated reporting module, generate_player_report.py. This script combines the outputs of multiple models into a structured Word document for a selected player.

Input

The script accepts a JSON file containing player information and feature sets required by the different models.

Output

The output is a formatted .docx scouting report containing:

- player overview;
- predicted goals and assists;
- injury risk assessment;
- predicted transfer market value;
- score summary table;

- final recommendation.

Scoring Logic

Raw model outputs are converted into normalized scores from 0 to 10. The system then averages the category scores and assigns a recommendation:

Overall Score	Recommendation
≥ 8.0	Strongly Recommended
≥ 6.5	Recommended
≥ 5.0	Conditional
< 5.0	Not Recommended

This module is important because it converts raw machine learning predictions into a form that scouts, coaches, and club executives can understand.

3.11 Data Visualization Techniques

The project also included interactive dashboards and visual analytics tools to support data exploration, model evaluation, and interpretation. These dashboards were developed for the main system components: the **Player Development and Squad Planning Engine**, the **Transfer Decision Engine**, and the **Match Simulation Engine**.

For the **Player Development and Squad Planning Engine**, the dashboard included dynamic scatter plots, feature distribution histograms, model comparison charts, actual-versus-predicted rating plots, feature-importance maps, and row-normalized confusion matrices. These visuals were used to explore player attribute patterns, compare classification and regression models, and identify position-classification errors.

For the **Transfer Decision Engine**, the visual outputs included injury-risk heatmaps, injury confusion matrices, player radar charts, performance correlation heatmaps, transfer value heatmaps, scouting report summaries, and interactive dashboards. These tools supported the interpretation of injury risk, next-season performance, transfer valuation, and final transfer recommendations.

For the **Match Simulation Engine**, the dashboard included shot classification outputs, shot feature-importance charts, match winner accuracy visuals, and live prediction charts. These visuals helped explain shot quality, evaluate match prediction performance, and support interactive match simulation inside the FIFA Career Mode system.

Together, these dashboards and visualizations made the system more interpretable and easier to use, allowing users to understand not only the final predictions, but also the patterns and model behavior behind them.

4.Results

This section presents the main findings produced by the implemented models and visual outputs. The interpretation focuses on model performance, patterns discovered in the data, and how each output supports football club decision-making.

4.1 Player Position Classification and Overall Rating Prediction

The player modeling results showed strong performance for both **position classification** and **overall rating prediction**.

For position classification, **Logistic Regression** achieved the best result with **82.82% accuracy**, outperforming KNN and Gaussian Naive Bayes.

Model	Task	Performance
Logistic Regression	Position classification	82.82% accuracy
K-Nearest Neighbors	Position classification	70.25% accuracy
Gaussian Naive Bayes	Position classification	53.71% accuracy

This result suggests that the selected player attributes were effective for identifying player roles. The gap between Logistic Regression and Naive Bayes also suggests that player attributes are not independent from each other, since football skills such as acceleration, sprint speed, dribbling, and ball control are naturally related.

For overall rating prediction, **Polynomial Regression, Degree 2** achieved the best performance.

Model	Task	R ² Score	MSE
Polynomial Regression, Degree 2	Overall rating prediction	0.9801	0.962
Ridge Regression	Overall rating prediction	0.8941	5.127
Linear Regression	Overall rating prediction	0.8941	5.128

The Polynomial Regression model produced a near-perfect fit on the test set, indicating that player overall ratings are strongly explained by combinations and interactions between technical and physical attributes.



Figure 1.FC 24 machine learning results dashboard showing classification accuracy, regression R^2 score, regression MSE, predicted versus actual rating, and influential player attributes.

4.2 Explainable Player Rating Analysis

The explainability analysis identified the attributes that contributed most strongly to the overall rating prediction.

The most important feature was **reactions**, with a relative importance score of **0.759**. This was much higher than the second-ranked feature, **ball control**, which had an importance score of **0.067**.

Attribute	Relative Importance
Reactions	0.759
Ball Control	0.067
Standing Tackle	0.021
GK Positioning	0.019
Jumping	0.014

This finding shows that the rating model is not only accurate, but also interpretable. In practical club terms, the model highlights reactions as a key indicator of elite player quality, while ball control and selected defensive or goalkeeper-related attributes contribute smaller but still meaningful signals.

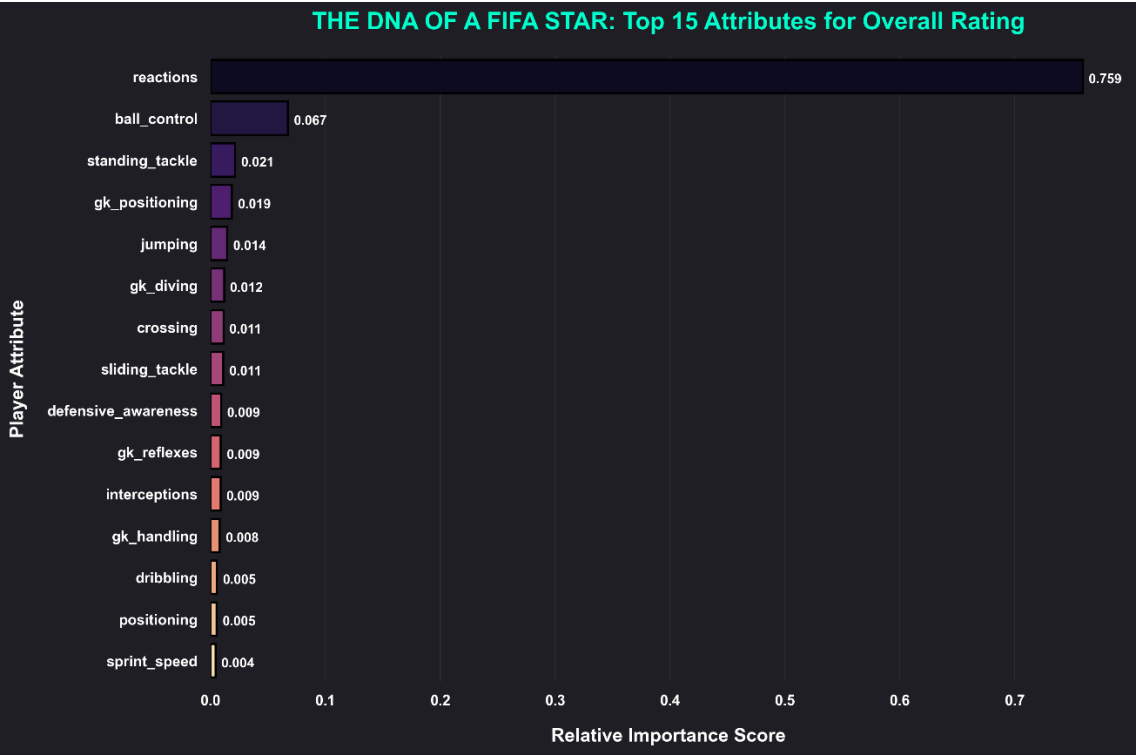


Figure 2. Feature-importance analysis for overall rating prediction. Reactions is the dominant predictor, followed by ball control and other technical, defensive, and goalkeeper-related attributes.

4.3 Injury Risk Prediction

The injury risk module produced two important outputs: a correlation analysis of injury-related factors and a confusion matrix showing classification performance.

The strongest correlations with next-season injury risk were associated with stress, sleep, balance, sprint speed, reaction time, knee strength, hamstring flexibility, and previous injuries.

Feature	Correlation with Injury Risk
Stress Level Score	0.53
Sleep Hours Per Night	0.51
Balance Test Score	0.49

Sprint Speed 10m	0.47
Reaction Time	0.47
Knee Strength Score	0.45
Hamstring Flexibility	0.45
Previous Injury Count	0.38

The results suggest that injury risk is influenced by a combination of physical condition, recovery, lifestyle, and injury history. For a club, this means player health assessment should not depend only on age or previous injuries, but also on broader physical and recovery indicators.

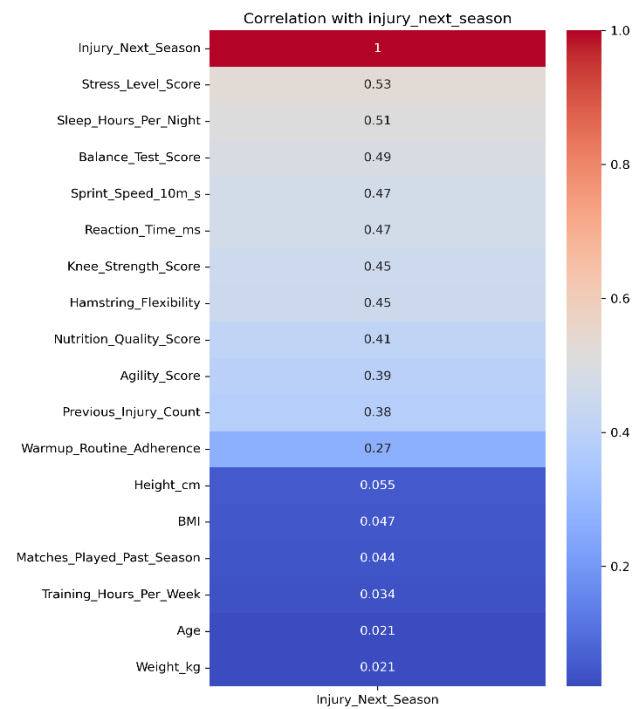


Figure 3. Correlation heatmap for next-season injury risk. Stress level, sleep hours, balance, sprint speed, reaction time, knee strength, hamstring flexibility, and previous injury count show the strongest relationships with injury risk.

The injury classifier achieved strong test performance. From the confusion matrix, the model correctly classified **152 out of 160** cases.

Actual / Predicted	No Injury	Injury
No Injury	75	5

Injury	3	77
--------	---	----

The calculated performance was:

Metric	Value
Accuracy	95.0%
Injury Precision	93.90%
Injury Recall	96.25%

The high injury recall is especially important because it means the model successfully identified most players who were actually at injury risk. In a club environment, this reduces the risk of signing or depending on physically risky players without warning.

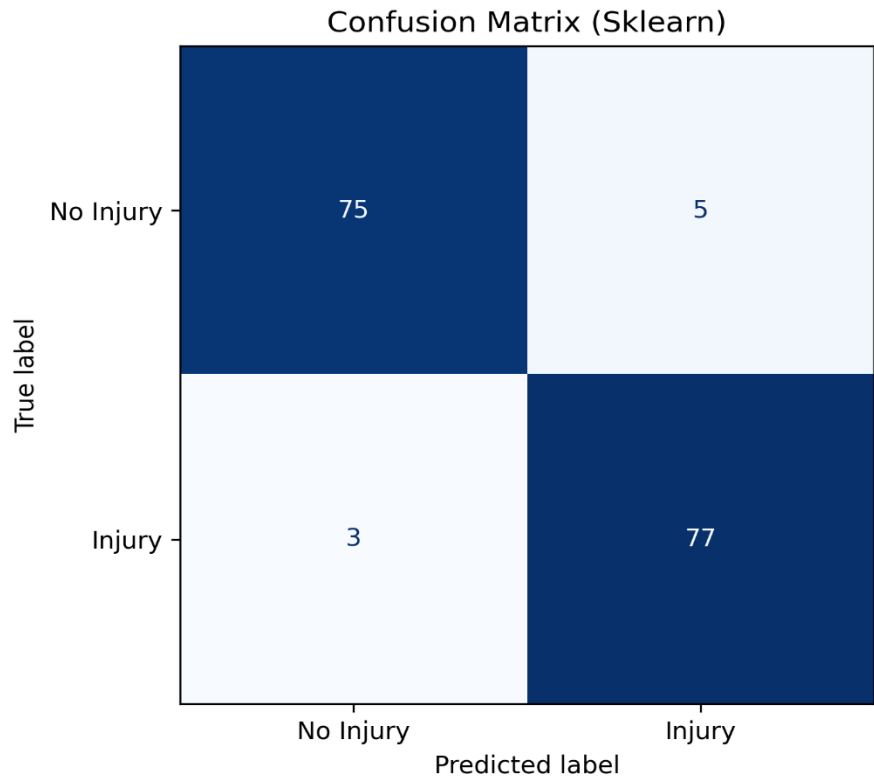


Figure 4. Confusion matrix for the injury risk classifier. The model correctly classified 152 out of 160 test cases, showing strong performance for both injury and no-injury classes.

A radar chart was also generated to summarize a selected player’s physical and health profile. This gives scouts and medical staff a quick visual summary of multiple risk-related indicators.

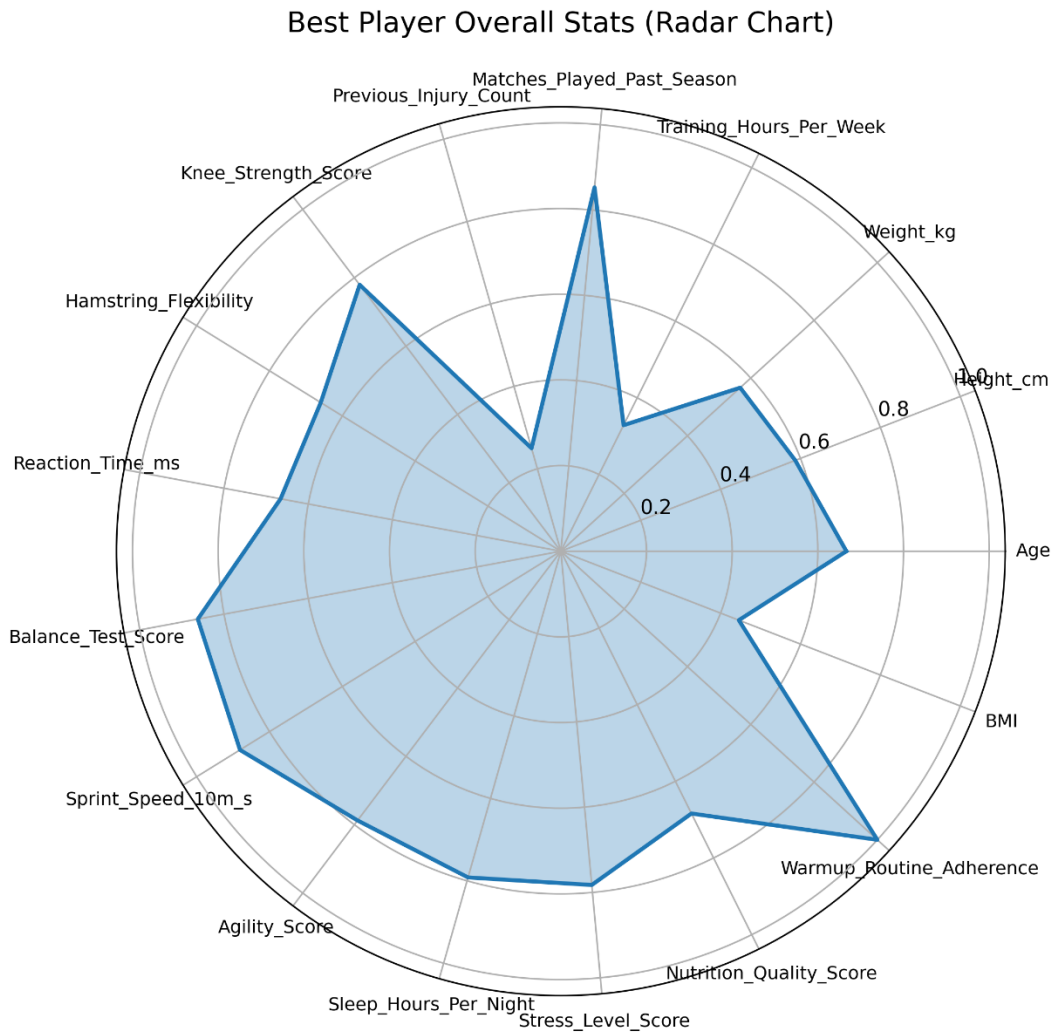


Figure 5. Radar chart showing a normalized physical and health profile for a selected player. The chart supports quick comparison of injury-related indicators during scouting or medical review.

4.4 Future Performance Prediction

The future performance module analyzed the variables most related to next-season goals and assists.

For **next-season goals**, the strongest correlations were mainly expected-goals and shooting-volume indicators.

Feature	Correlation with goals_next
Expected xG	0.65
Expected npxG	0.65
Standard Goals	0.63

Performance Goals	0.63
Shots on Target	0.63
Expected npxG + xAG	0.62
Touches in Attacking Penalty Area	0.62

These results show that future goal output is strongly connected to chance quality, previous scoring output, shots on target, and penalty-area involvement.

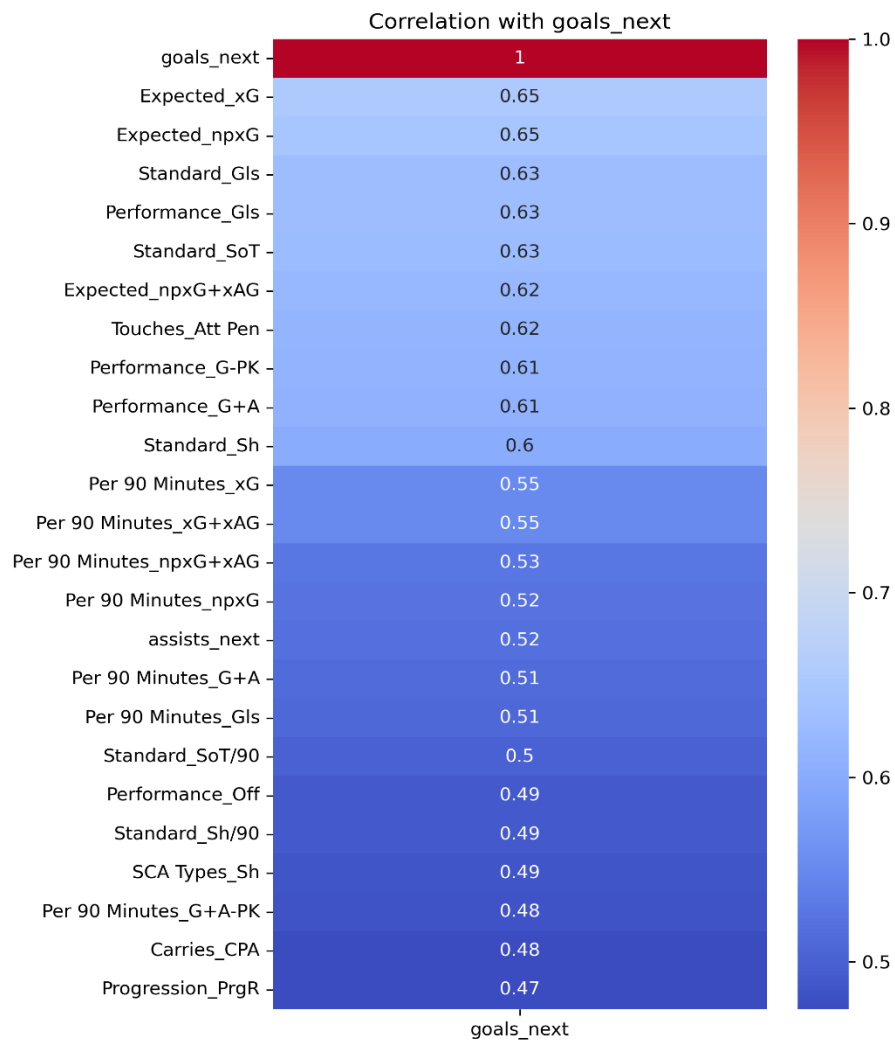


Figure 6. Correlation heatmap for next-season goals prediction. Expected-goals metrics, previous goals, shots on target, and penalty-area touches show the strongest relationships with future goal output.

For **next-season assists**, the strongest correlations were creative and attacking involvement indicators.

Feature	Correlation with assists_next
Expected xA	0.53
xAG	0.53
Expected xAG	0.53
Touches in Attacking Third	0.52
Shot-Creating Actions	0.52
Goals Next	0.52
Key Passes	0.51
Goal-Creating Actions	0.51

These results show that future assists are linked to chance creation, key passing, attacking-third activity, and involvement in goal-creating actions.

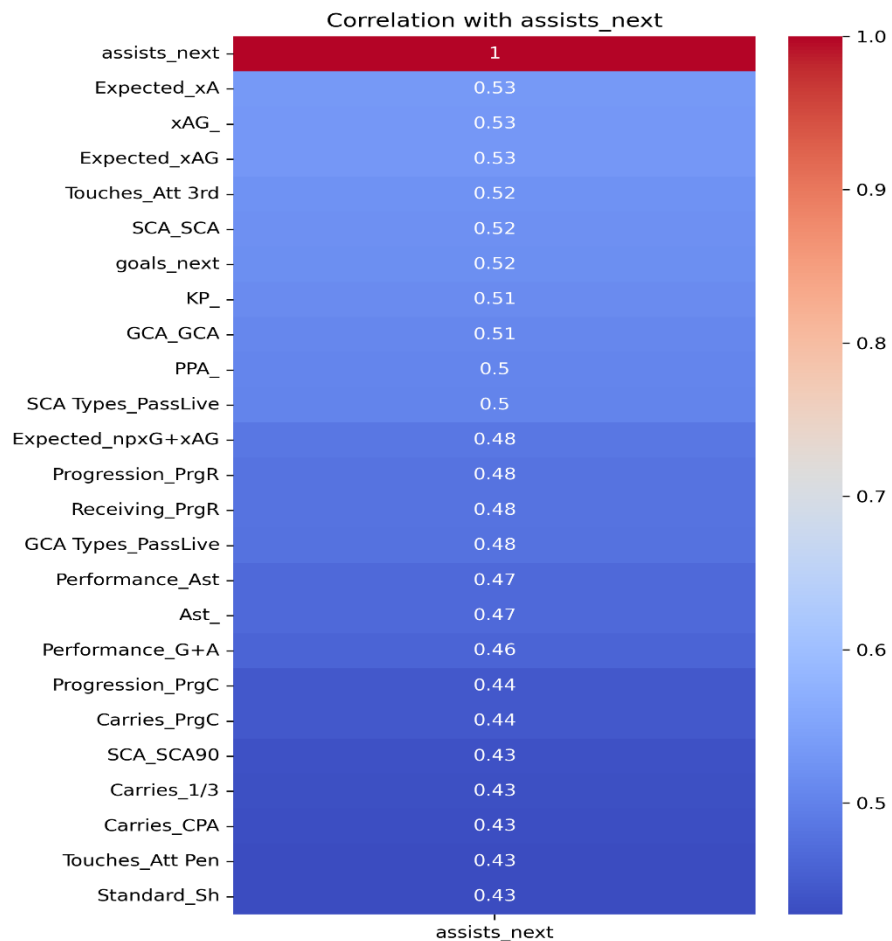


Figure 7. Correlation heatmap for next-season assists prediction. Creative metrics such as expected assists, key passes, shot-creating actions, goal-creating actions, and attacking-third touches are the strongest indicators of future assists.

4.5 Transfer Market Value Prediction

The transfer value results showed that market value was most strongly related to historical value and club context.

Feature	Correlation with Current Value
Highest Value	0.83
Team Encoded	0.68
Minutes Played	0.42
Appearance	0.42
Award	0.30
Assists per 90	0.17
Games Injured	0.14
Goals per 90	0.13

The strongest variable was **highest_value**, indicating that historical peak value is a major signal for current value. The second strongest feature was **team_encoded**, showing that club context also has a strong relationship with market valuation. Playing-time variables such as minutes played and appearances provided moderate predictive signals.

For club decision-making, this result is useful because it shows that transfer value is not based only on goals and assists. Reputation, club level, playing involvement, and previous market valuation all influence estimated player value.

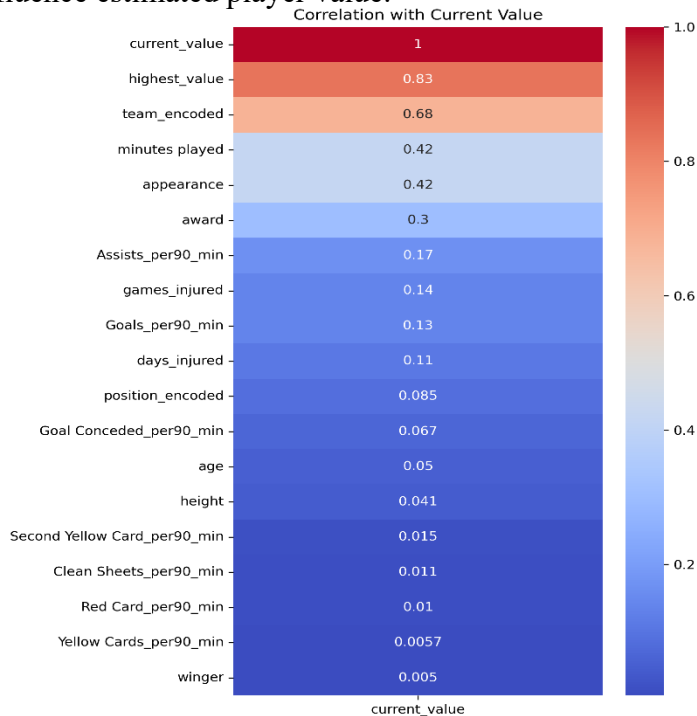


Figure 8. Correlation heatmap for transfer market value prediction. Historical peak value and team-level encoding show the strongest relationships with current market value.

4.6 Automated Player Scouting Report

The automated scouting report successfully combined model outputs into a single decision-support document.

An example report was generated for **Marcus Fernandez**, a 26-year-old forward from Atletico Madrid. The report produced the following outputs:

Category	Output
Predicted Goals Next Season	11.0
Predicted Assists Next Season	6.0
Predicted Goals + Assists	17.0
Injury Probability	53.2%
Predicted Market Value	€14,970,376
Overall Score	5.75 / 10
Final Recommendation	Conditional

The final recommendation was **Conditional**. The player showed good predicted attacking output, but the injury probability was relatively high and the market value score was moderate. Therefore, the system recommended further review of the player’s injury history and tactical fit before making a transfer decision.

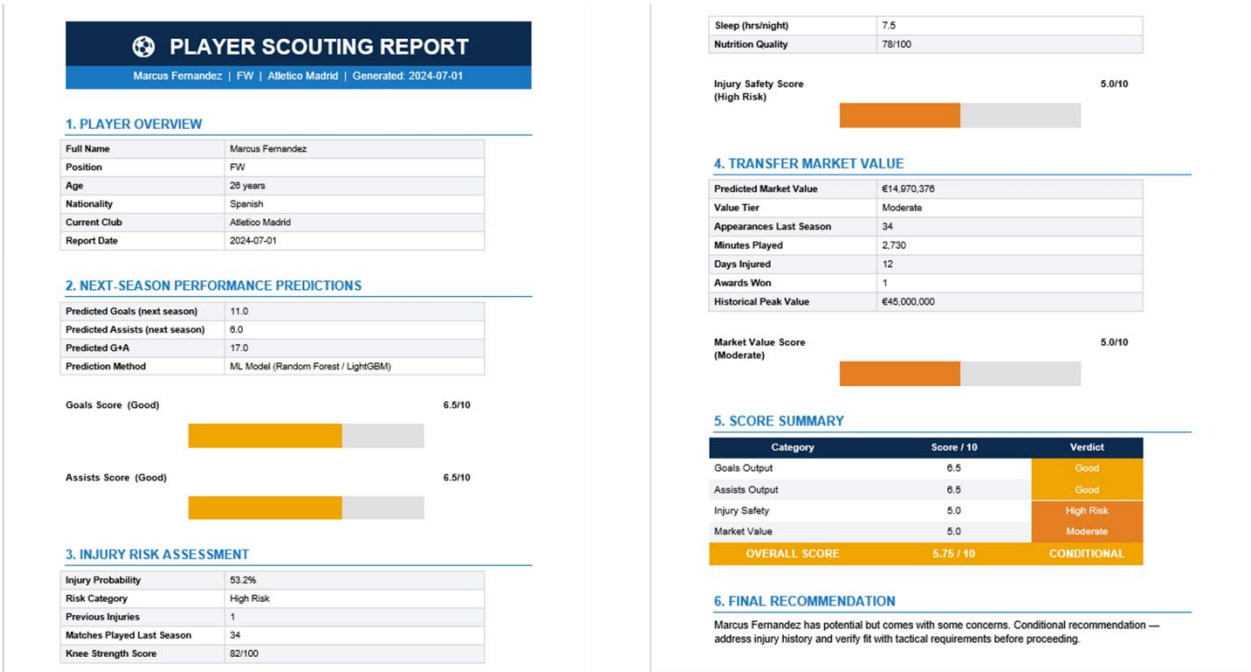


Figure 9. Example automated scouting report generated by the system. The report combines performance prediction, injury risk assessment, transfer value estimation, score summaries, and a final recommendation.

4.7 Match Simulation Engine

- The **Match Simulation Engine** represents the match-day component of the FIFA Career Mode system. It supports fixture simulation by predicting shot outcomes, expected scorelines, and likely match winners. The results are divided into two main parts: **shot outcome classification** and **match score/winner prediction**.

1. Shot Outcome Classification Results

The shot classification model predicts whether a selected shot is likely to become a goal based on shot distance, shooting angle, body part, and pressure. In the dashboard example, a shot from **12 meters**, with a **45-degree angle**, taken by **foot** and under **no pressure**, was predicted as a **goal** with a probability of **72.3%**.

The confusion matrix showed an overall shot classification accuracy of **89.7%**.

Actual / Predicted	Predicted No Goal	Predicted Goal
Actual Goal	1,641	311
Actual No Goal	15,475	171

The model correctly classified a large number of no-goal shots, which is important because most football shots do not result in goals. However, the matrix also shows that the model missed some actual goals, which is expected because goal events are much rarer and harder to classify than missed shots.

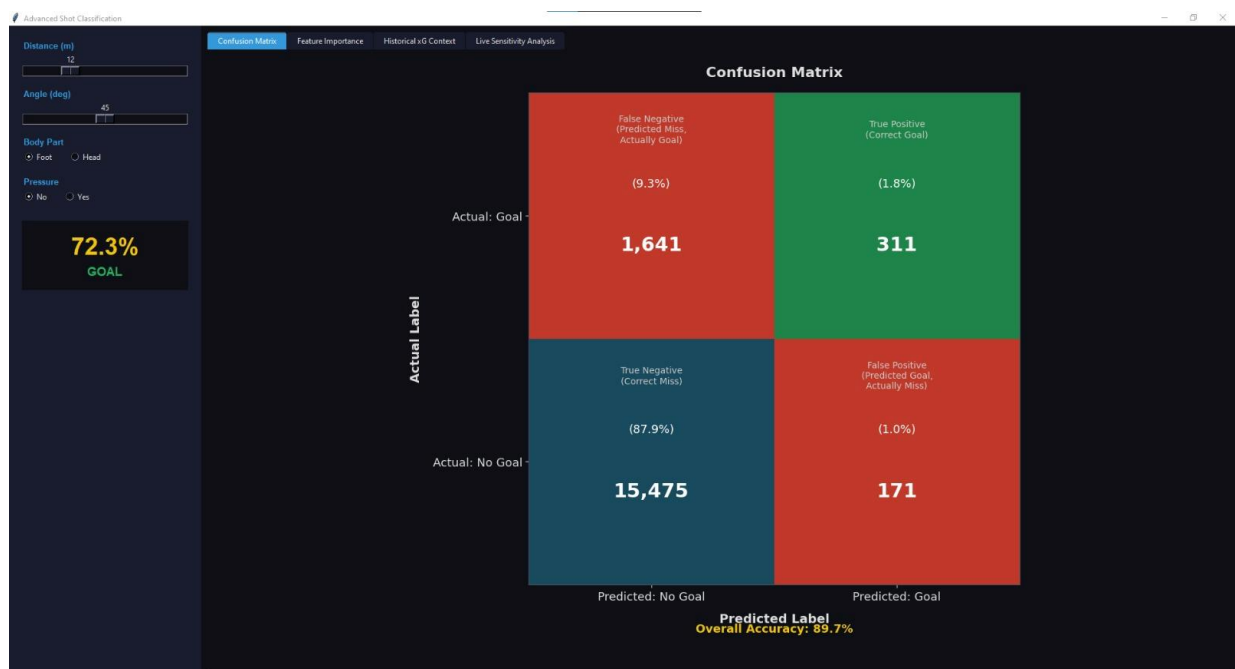


Figure 10. Confusion matrix for the shot outcome classifier. The model achieved 89.7% overall accuracy, with strong no-goal classification performance.

The feature-importance analysis showed that **shot angle** was the most influential variable, contributing **53.1%** of the model’s decision weight. **Distance** contributed **22.9%**, while **body part** contributed **22.4%**. **Pressure** had the smallest effect, with only **1.6%** importance.

Feature	Relative Importance
Angle	53.1%
Distance	22.9%
Body Part	22.4%
Under Pressure	1.6%

This result suggests that the quality of the shooting angle is the strongest factor in determining whether a shot is likely to become a goal. In FIFA Career Mode, this helps the user understand which chances are actually dangerous, instead of judging shots only by distance or visual appearance.

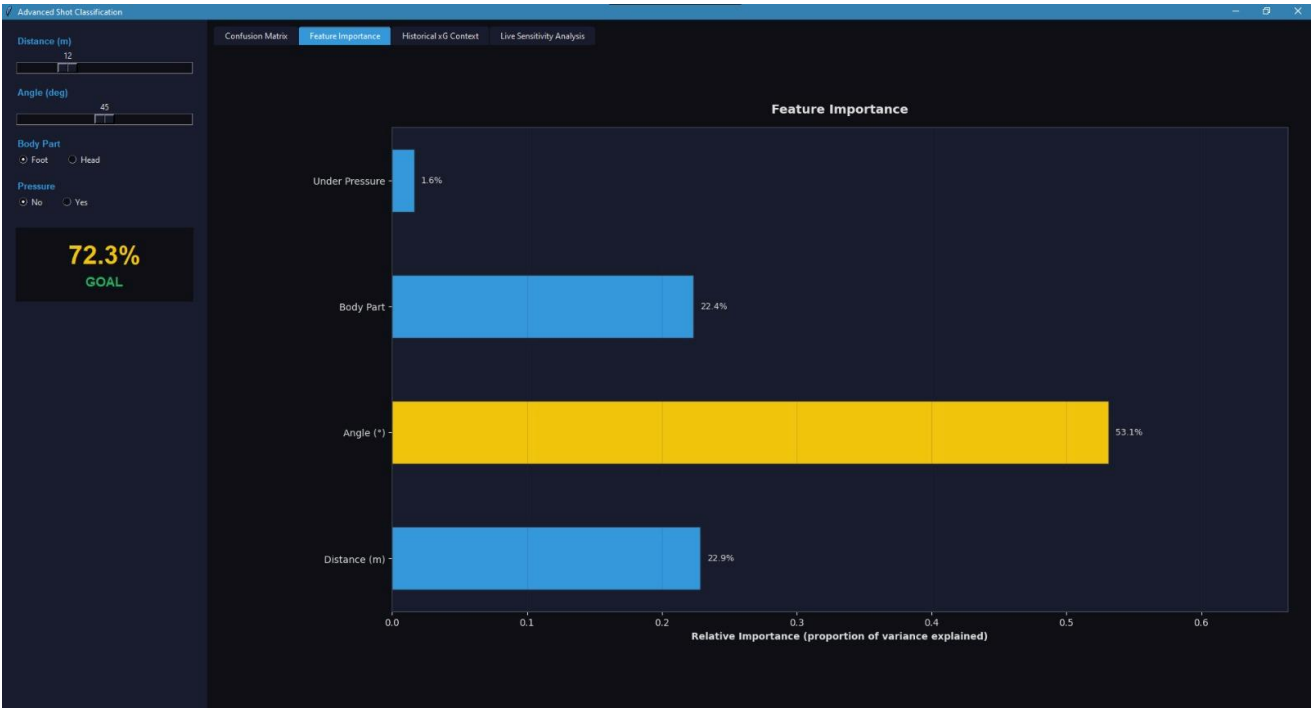


Figure 11. Feature importance output from the advanced shot classification dashboard. Shot angle is the most influential feature, followed by distance and body part.

2. Match Score and Winner Prediction Results

The match prediction model estimates home goals, away goals, goal difference, and the likely winner. In the dashboard example, the model produced the following output:

Output	Value
Predicted Home Goals	1.89
Predicted Away Goals	1.11
Goal Difference	+0.78
Predicted Winner	Home Win
Home Win Probability	59%
Away Win Probability	41%

This output supports the FIFA Career Mode simulation theme because the user can estimate the expected match result before choosing whether to simulate or manually play a fixture.

The model performance summary showed:

Metric	Value
Home Goals R^2	0.1781
Away Goals R^2	0.1170
RMSE	1.3952
Winner Accuracy	51.7%

The R^2 values show that exact scoreline prediction is difficult, which is expected in football because scores are low-count, random, and affected by many match events. However, the model still provides a useful baseline by estimating expected goals and converting them into a predicted match outcome.

Winner prediction performance differed strongly by outcome class.

Outcome Class	Prediction Accuracy
Home Win	88.6%
Draw	3.3%
Away Win	36.3%
Overall	51.7%

The model performed best when predicting **Home Wins**, reaching **88.6%** accuracy for that class. Draws were the hardest outcome, with only **3.3%** accuracy. This is a useful limitation because draws are naturally difficult to predict and often depend on small match events.

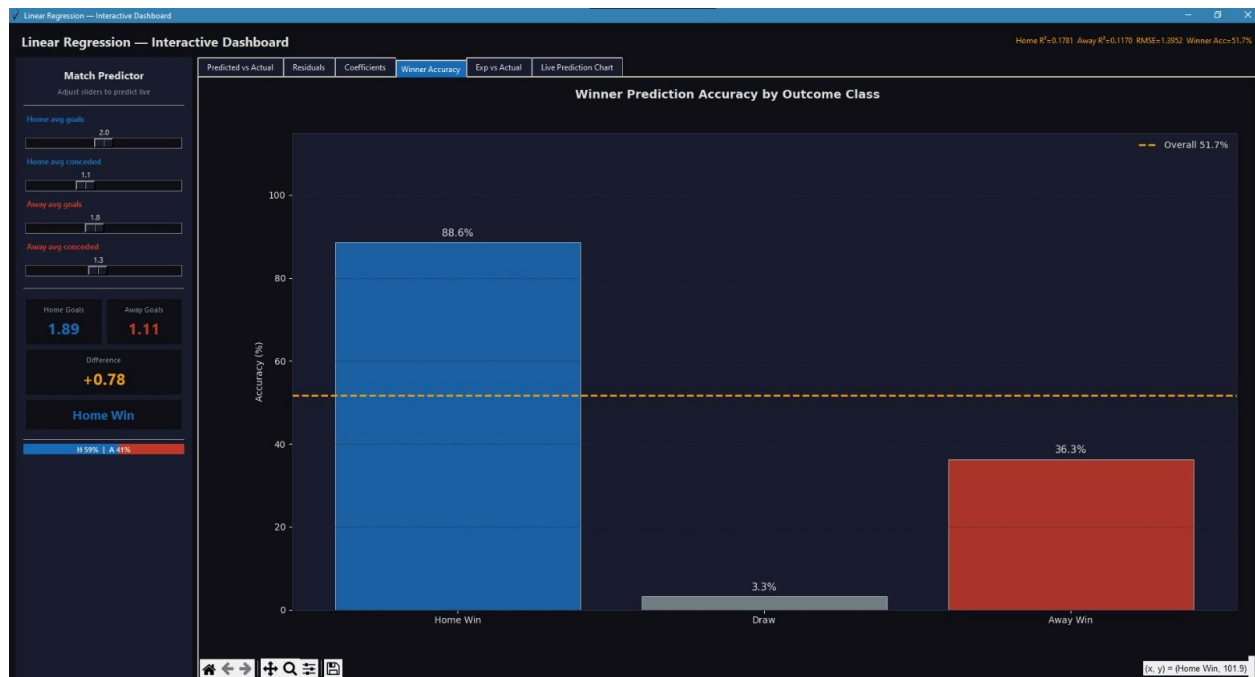


Figure 12. Winner prediction accuracy by outcome class. The model performed best on home wins, while draws were the most difficult class to predict.

The live prediction chart also showed how changing average home and away goal inputs affects the predicted scoreline. This interactive feature is useful for FIFA Career Mode because the user can adjust team-strength assumptions and immediately observe how the predicted result changes.

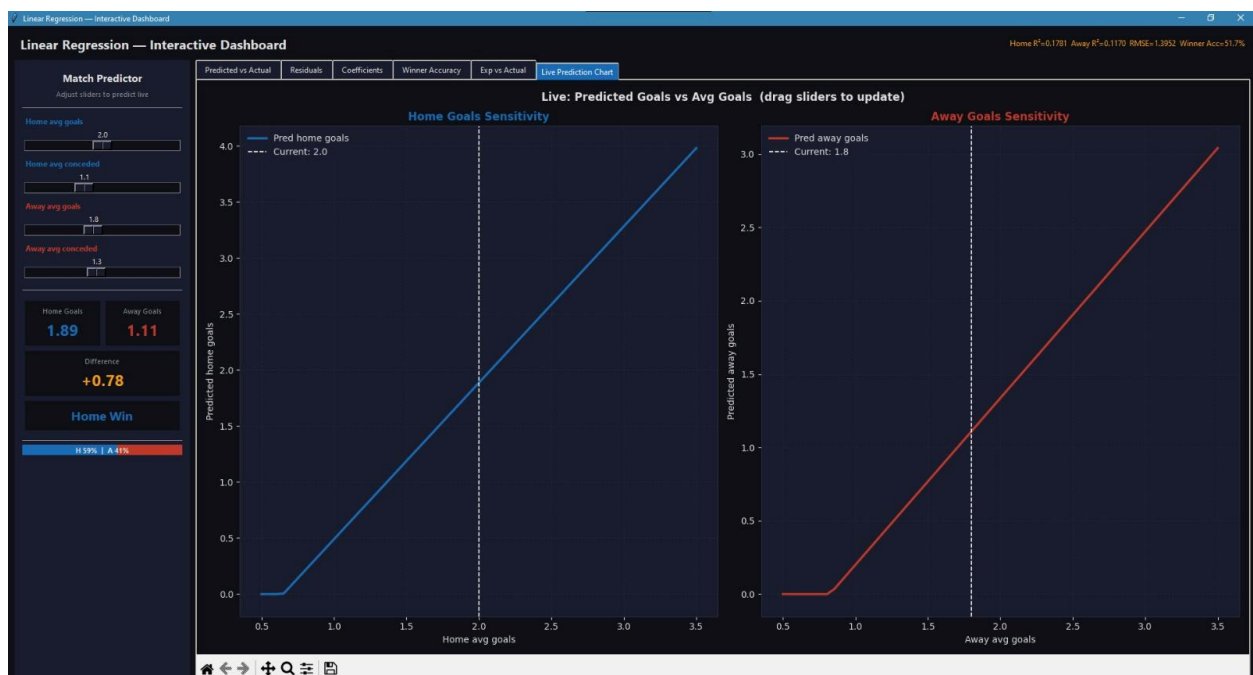


Figure 13. Live prediction chart showing how home and away average goals affect predicted scorelines.

3. Interpretation for FIFA Career Mode

The Match Simulation Engine adds the match-day layer to the overall system. While the other modules focus on player development, squad planning, injury risk, and transfer decisions, this module helps the user simulate fixtures and understand match situations.

The shot classifier is useful because it evaluates chance quality. Its results showed that **angle**, **distance**, and **body part** were the most important factors in predicting whether a shot becomes a goal. The match predictor provides a broader fixture-level output by estimating scoreline and likely winner.

However, the results also show that match prediction is more uncertain than some player-level tasks. The winner accuracy of **51.7%** and the low draw accuracy show that football outcomes are difficult to predict using simple match-level features. Therefore, this engine should be used as a **simulation assistant**, not as a perfect result predictor

Overall, this module strengthens the FIFA Career Mode system by adding support for:

- match simulation;
- shot quality evaluation;
- scoreline prediction;
- winner prediction;
- interactive scenario testing.

5. Conclusion

5.1 Summary of Findings

This project developed an **AI-powered FIFA Career Mode decision-support system** that combines multiple machine learning models to support match simulation, player development, squad planning, injury assessment, performance forecasting, and transfer evaluation. Instead of treating each model as a separate task, the project connects them into one system that helps a Career Mode user make smarter decisions when managing a virtual football club.

The **Player Development and Squad Planning Engine** showed strong results in predicting player position and overall rating. Logistic Regression achieved the best position-classification performance with **82.82% accuracy**, while Polynomial Regression achieved the strongest overall rating prediction performance with **$R^2 = 0.9801$** and **$MSE = 0.962$** . The player recommender model also added practical value by identifying statistically similar players, which can help users find replacements or alternative transfer targets.

The explainability analysis showed that **reactions** was the most influential attribute behind overall player rating, with a feature-importance score of **0.759**. This insight is useful because it

shows that the system does not only make predictions, but also helps explain which attributes matter most in player evaluation.

The **Transfer Decision Engine** produced useful outputs for evaluating potential signings. The injury risk classifier achieved **95.0% accuracy** and **96.25% injury recall**, meaning it was effective at identifying risky players. The performance prediction analysis showed that future goals were strongly related to expected-goals metrics, previous goals, shots on target, and penalty-area touches, while future assists were linked to expected assists, key passes, shot-creating actions, and attacking-third involvement. The transfer value analysis showed that current market value was most strongly related to historical peak value, team context, minutes played, and appearances.

The automated scouting report successfully combined predictions into a final recommendation, showing how the system can translate raw model outputs into a decision-ready format for Career Mode transfer planning.

The **Match Simulation Engine** is intended to support fixture simulation by predicting match winners, scorelines, shot outcomes, and scoring-related events.

5.2 Recommendations

Based on the results, the system can be used in FIFA Career Mode to support several practical decisions:

- **Use the Match Simulation Engine before important fixtures.**
The user can use the match prediction outputs to estimate the expected scoreline, likely winner, and scoring events. This can help with tactical preparation, squad rotation, and deciding whether a match should be simulated or played manually.
- **Use the shot outcome and scorer-related predictions to analyze attacking chances.**
These outputs can help identify which types of chances are most dangerous and which players are more likely to score. This supports match analysis and improves decision-making during Career Mode simulations.
- **Use the position prediction model before assigning player roles.**
If a player's listed position does not match his attribute profile, the model can suggest a more suitable role.
- **Use the overall rating prediction model to evaluate player quality.**
The rating prediction model can help estimate whether a player's attributes justify his current rating or expected development.

- **Use the player recommender when selling or replacing players.**
When a user sells a key player, the recommender can identify similar players with comparable attribute profiles.
- **Use injury risk before completing transfers.**
A player with high predicted performance but high injury risk should be treated as a risky signing.
- **Use performance prediction to judge attacking value.**
Predicted goals and assists can help decide whether a player is likely to contribute enough in future seasons.
- **Use predicted transfer value to avoid overpriced signings.**
If the model's predicted value is much lower than the listed market price, the transfer should be reviewed carefully before proceeding.
- **Use the automated report for final transfer decisions.**
The report is useful because it combines performance, injury risk, market value, and scoring logic into one final recommendation.

Overall, the system should not replace user judgment completely. It should act as a **decision-support assistant** that gives data-driven evidence before the user makes the final Career Mode decision.

5.3 Limitations

Although the project produced strong results, several limitations should be considered.

First, the datasets came from different sources, including StatsBomb open data, EA Sports FC 24 player data, and Kaggle football datasets. Because these datasets were not originally designed as one unified system, some features are not perfectly aligned across modules. This means that full integration into one application would require additional feature matching and data standardization.

Second, some datasets represent real football data, while others are based on FIFA-style player attributes or public Kaggle datasets. As a result, the system may not perfectly match the internal mechanics of FIFA Career Mode in every case.

Third, some models may be affected by dataset-specific patterns. For example, the overall rating model achieved a very high R^2 score, which suggests strong predictive ability, but it may also reflect the structured nature of FIFA player ratings. Therefore, the result should be interpreted carefully.

Fourth, the transfer value model estimates value from available statistical features, but real and in-game transfer values can also be affected by factors such as contract length, player potential,

release clauses, wage demands, age curve, club negotiation behavior, and popularity. These factors were not fully included.

Fifth, the injury and performance models depend on historical data patterns. Player development in Career Mode can be affected by gameplay decisions, training plans, morale, form, and user-controlled match time, which may not be fully captured in the available datasets.

Finally, the Match Simulation Engine showed that predicting football match outcomes is more difficult than predicting player-level attributes or injury risk. Although the shot classifier achieved strong performance with **89.7% accuracy**, the match winner model achieved a more limited overall accuracy of **51.7%**, with especially weak draw prediction. This limitation suggests that match simulation should be used as a supportive estimation tool rather than a perfect predictor of FIFA Career Mode results.

5.4 Future Work

Future work can improve the system in several directions.

The transfer module can be strengthened by adding more Career Mode-specific variables, such as:

- player potential;
- wage;
- contract length;
- release clause;
- age and development curve;
- morale;
- form;
- playing time;
- club budget;
- transfer listing status.

The player recommender can also be improved by adding filters such as age, value, wage, league, nationality, position, potential, and budget range. This would make the recommendations more realistic for Career Mode transfer decisions.

The Match Simulation Engine can be extended by predicting not only the winner and scoreline, but also:

- assist providers;

- expected shots;
- expected possession;
- player ratings after the match;
- probability of clean sheets;
- match difficulty level.

The Match Simulation Engine can also be improved by addressing the weak draw prediction and moderate overall winner accuracy. Future versions could use richer match-level features such as team form, player ratings, lineup strength, injuries, fatigue, home advantage, tactical style, and recent scoring trends. More suitable football score models, such as Poisson Regression, Random Forest, XGBoost, or ensemble models, could also be tested to improve scoreline and winner prediction, especially for draws and close matches.

Model evaluation can also be improved by testing on multiple seasons, comparing more algorithms, and using more advanced models such as XGBoost, LightGBM, CatBoost, or neural networks.

Finally, the system could be improved by allowing users to import live Career Mode saves or manually enter squad data directly from their own gameplay. This would make the predictions more personalized to each user’s Career Mode team and season context.

6. Acknowledgements

6.1 Team Contributions

This project was completed through the combined work of all team members, with each subgroup contributing a different part of the **AI-powered FIFA Career Mode decision-support system**.

Team Members	Contribution
Zaytoun & Hesham	Developed the Match Simulation Engine using StatsBomb match-event data. Their work included preparing match and shot data, building models for shot outcome classification, scoreline prediction, and winner prediction, and creating the related dashboard with shot classification outputs, feature-importance visuals, winner accuracy results, and live match prediction charts.

Abdelrahman & Ziad	Developed the Player Development and Squad Planning Engine using the EA Sports FC 24 player dataset. Their work included data cleaning, duplicate and leakage detection, position classification, overall rating prediction, player recommendation, explainability analysis, dashboard creation, and contribution to organizing and writing the technical report.
Gomaa & Seif	Developed the Transfer Decision Engine using injury, performance, and transfer-value datasets. Their work included injury risk classification, next-season goals and assists prediction, transfer market value prediction, automated scouting report generation, and interactive dashboards for reviewing player risk, performance, value, and final transfer recommendations.

Overall, the team integrated these contributions into one coherent system that supports Career Mode users in match simulation, squad planning, player evaluation, transfer analysis, and final decision-making.

6.2 External Support and Resources

The team acknowledges the external datasets, tools, and libraries that supported the implementation of this project.

Resource	Usage
StatsBomb Open Data	Used as the source of match-event data for match simulation, shot outcome prediction, scoreline prediction, and winner prediction.
Kaggle Datasets	Used for player attributes, injury prediction, performance forecasting, and transfer market value prediction.
EA Sports FC 24 Player Dataset	Used for position classification, overall rating prediction, player recommendation, and explainability analysis.
Python	Main programming language used for data processing, modeling, visualization, and report generation.
pandas	Used for loading, cleaning, transforming, and exporting datasets.
NumPy	Used for numerical calculations, feature engineering, and mathematical operations.
scikit-learn	Used for machine learning models, preprocessing pipelines, train-test splitting, GridSearchCV, cross-validation, and evaluation metrics.

Matplotlib and Seaborn	Used for producing heatmaps, confusion matrices, radar charts, feature-importance plots, and other visualizations.
statsbombpy	Used to access StatsBomb open football data programmatically.
joblib	Used for saving and loading trained machine learning models and encoders.
python-docx	Used for generating the automated player scouting report in Word document format.

The team also acknowledges the role of publicly available documentation for the libraries and datasets used throughout the project.

7. References

“EA Sports FC 24 Complete Player Dataset.” *Kaggle*, uploaded by Stefano Leone, www.kaggle.com/datasets/stefanoleone992/ea-sports-fc-24-complete-player-dataset/data?select=male_players.csv.

“Football Players Transfer Fee Prediction Dataset.” *Kaggle*, uploaded by Khang Huynh Nguyen Truong, www.kaggle.com/datasets/khanghunhnguyentrng/football-players-transfer-fee-prediction-dataset.

“Matplotlib: Visualization with Python.” *Matplotlib*, matplotlib.org/.

“Random Forest Regression in Python.” *GeeksforGeeks*, www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/.

“scikit-learn: Machine Learning in Python.” *scikit-learn*, scikit-learn.org/stable/.

StatsBomb. “Open Data.” *GitHub*, github.com/statsbomb/open-data.

“Top 5 League Football Player Stats 2017–2025.” *Kaggle*, uploaded by Emre Yilmaz, www.kaggle.com/datasets/emrey3lmaz/top-5-league-football-player-stats-2017-2025.

“University Football Injury Prediction Dataset.” *Kaggle*, uploaded by Yuanchun Hong, www.kaggle.com/datasets/yuanchunhong/university-football-injury-prediction-dataset.

9. Appendix A: AI Usage and Audit Trail

A.1 Purpose of AI Use

AI tools were used as technical development assistants during the implementation of the project. The team used AI to support coding, dashboard development, GUI implementation, data

cleaning, visualization creation, GitHub data fetching, README preparation, and automated report generation.

Three AI tools were mainly used: **Claude**, **ChatGPT**, and **Gemini**. Each tool supported different parts of the implementation process. The team used the AI outputs as technical guidance and development support, then reviewed, tested, modified, and integrated the final work into the project.

A.2 AI Tools Used

AI Tool	Main Technical Use
Claude	Assisted with implementing the automated player report generator, building GUI dashboards, combining visual outputs into dashboards, helping with README file creation, supporting StatsBomb/GitHub data fetching, creating visuals for the Match Simulation Engine, and supporting the combined dashboard.
ChatGPT	Assisted with data cleaning, Python syntax support, code explanation, visualization guidance, and general implementation support.
Gemini	Assisted with creating visuals for Player Development and Squad Planning Engine, especially model comparison visuals, rating prediction visuals, and player analytics charts. Gemini also helped with data-cleaning support during the EA Sports FC 24 player modeling process.

A.3 Human Review and Technical Control

AI was used as a technical support tool during development, but the final project was controlled and reviewed by the team. The team did not use AI outputs without checking them. All AI-assisted code, dashboards, visualizations, report-generation logic, and README content were reviewed, tested, edited, and integrated manually.

The team maintained technical control by:

- applying AI suggestions to the actual project datasets;
- testing generated code and dashboard behavior;
- checking that model outputs appeared correctly in the dashboards;
- validating visualizations against real model results;
- reviewing the generated player report and its recommendation logic;
- correcting implementation details where AI output did not fully match the project;
- ensuring that the final system matched the FIFA Career Mode project theme.

A.4 AI Chat Links

The following links provide access to the AI chat sessions used during the technical development process.

1. [Claude1](#)
2. [Claude2](#)
3. [Chatgpt](#)
4. [Gemini](#)

A.5 Audit Trail Screenshots

1. Project improvement helper

i have data science project and i am prediciting the football players goals and assists for the next season the problem is that i used randomforest and xgboost algorithms and both of them gave me low results as the r2_score of goals_next was 0.53 and assists_next was 0.43 so how to improve the performnce

Let me give you a comprehensive breakdown of how to improve your model performance.

Why R² is Low for Player Stats Prediction

Football stats are inherently noisy — injuries, team changes, manager tactics, and luck all contribute. But there's a lot you can do to squeeze more signal out of your data.

1. 🛠️ Feature Engineering (Biggest Impact)

This is likely your #1 bottleneck. Raw stats aren't enough.

Rolling/Lagged Features

```
python
df['goals_last3_avg'] = df.groupby('player_id')['goals'].transform(
    lambda x: x.rolling(3, min_periods=1).mean()
)
df['goals_lag1'] = df.groupby('player_id')['goals'].shift(1)
df['goals_lag2'] = df.groupby('player_id')['goals'].shift(2)
df['assists_lag1'] = df.groupby('player_id')['assists'].shift(1)
```

Per-90 Minute Stats (normalizes for playing time)

```
python
df['goals_per90'] = (df['goals'] / df['minutes_played']) * 90
df['assists_per90'] = (df['assists'] / df['minutes_played']) * 90
df['shots_per90'] = (df['shots'] / df['minutes_played']) * 90
```

Efficiency & Conversion Rates

```
python
df['shot_conversion'] = df['goals'] / df['shots'].replace(0, 1)
df['key_pass_conversion'] = df['assists'] / df['key_passes'].replace(0, 1)
df['xG_overperformance'] = df['goals'] - df['xG'] # luck indicator
df['xA_overperformance'] = df['assists'] - df['xA']
```

2. position/overall rating predictor visuals

i want to make sure it runs on any device not just my device

2. 📊 Add Better Input Features

These external factors dramatically improve predictions:

Feature	Why It Helps
<code>xG / xA</code>	Better than raw goals — removes luck
<code>minutes_played</code>	Accounts for rotation/injuries
<code>team_strength</code> (league table position)	Better team = more chances
<code>position</code> (encoded)	Strikers score more than defenders
<code>age + age²</code>	Non-linear career curve
<code>shots_on_target</code>	Leading indicator for goals
<code>progressive_carries</code> / <code>passes</code>	Chance creation proxy
<code>penalty_kicks_taken</code>	Easy goals inflator

I completely understand. It is annoying to have to remember a special `streamlit run` command when you are used to just hitting the "Run" button in your code editor or typing `python your_file.py`.

Because Streamlit is actually launching a local web server (rather than just running a script from top to bottom), it usually requires that special command. However, there is a very clever Python trick to bypass this so you can run it "normally."

You just need to add a few lines of code to the **very bottom** of your `rating-position-visuals.py` file.

How to fix it:

Open your `rating-position-visuals.py` file, scroll all the way down to the bottom, and paste this exact block of code:

Python



```
# — Run Normally Hack —————
if __name__ == '__main__':
    import sys
    from streamlit.web import cli as stcli

    # This tricks Python into running the Streamlit server command automatically
    sys.argv = ["streamlit", "run", __file__]
    sys.exit(stcli.main())
```

Why this works:

When you hit "Run" in your IDE or type `python rating-position-visuals.py`, Python looks at the bottom of the file. That block of code intercepts the normal run process and says, "Hey, pretend the user just typed `streamlit run` in the terminal for this exact file."

Now, you can run the file using your normal Python command:

Bash



```
python "d:/data science projjj/Football-Data-Analysis/src/rating-position-visuals.py"
```

Or simply click the "Run/Play" button in VS Code, PyCharm, or whatever editor you are using, and it will automatically pop open the interactive interface in your browser!

3. predict how many goals and is this shot a goal or not

and what type of regression/ classification are we going to use or is best fit for this data

🏠 Best ML Models for Your Football Project

📊 REGRESSION Models (predicting a number)

1. Random Forest Regressor ← Best Pick

- Predict: xG (Expected Goals) per match
- Why: Football data is non-linear and messy — Random Forest handles that perfectly without needing feature scaling. It also tells you **feature importance** (e.g., shot distance matters more than angle), which looks great in your report.
- Features to use: shot distance, shot angle, body part, game state (score at time of shot), match minute

2. Linear Regression ← Required Baseline

- Predict: Total goals scored by a team per season based on possession %, shots per game, pass accuracy
- Why: It's simple, interpretable, and required by the project. Use it as a baseline to compare against Random Forest and show why a more complex model is needed.

💡 CLASSIFICATION Models (predicting a category)

1. Logistic Regression ← Best Pick for Match Outcome

- Predict: Match result — Home Win / Draw / Away Win (3 classes)
- Why: It outputs **probabilities** (e.g., 60% home win, 25% draw, 15% away win), which feeds directly and beautifully into your **match simulator**. This is the bridge between your ML model and the simulator — you use these probabilities to drive the simulation.
- Features to use: home/away team average goals, xG, form (last 5 results), head-to-head record

2. Decision Tree Classifier ← Great Visual Model

- Predict: Whether a shot results in a Goal or No Goal
- Why: Decision trees are **highly visual and explainable** — you can literally draw the tree and show it in your dashboard. It pairs perfectly with the shot map visualization. Also easy to tune with `max_depth` to avoid overfitting.
- Features to use: xG value, shot location zone, body part, assist type, pressure on player

🔗 How Everything Connects

Here's the beautiful part — all 4 models feed into your dashboard and simulator in a chain:

